Đặc tả Kỹ thuật: Ứng dụng Blog Next.js

1. Thông tin dự án

1.1. Tên dự án

NextJS Blog Learning Platform

1.2. Mô tả

Ứng dụng blog cá nhân được xây dựng bằng Next.js 16 (App Router), phục vụ mục đích đào tạo và thực hành các kiến thức Next.js từ cơ bản đến nâng cao.

1.3. Mục đích Sử dụng

1.3.1. Demo Application cho Giảng viên

- ✅ Demo live các concept trong lớp
- ✅ Reference code chuẩn cho học viên
- ✅ Troubleshooting examples
- ✅ Deploy link: `https://nextjs-blog-demo.vercel.app` (example)

1.3.2. Learning Resource cho Học viên

- 📚 Starter template để practice
- 📚 Progressive commits để follow từng bước
- 📚 Code examples cho mỗi concept
- 📚 Exercises với solutions

1.3.3. Teaching Strategy

```
┌──────────────┐
│ Giảng viên   │ → Demo live trên ứng dụng đã build sẵn
└──────────────┘
       ↓
┌──────────────┐
│ Học viên     │ → Clone về và implement từng feature
└──────────────┘
       ↓
┌──────────────┐
│ Reference    │ → So sánh code của mình với solution
└──────────────┘
```

1.4. Mục tiêu Kỹ thuật

- Áp dụng đầy đủ các tính năng Next.js: Image, Routing, Server/Client Components, Data Fetching
- Thực hành form handling và validation
- Hiểu rõ sự khác biệt giữa SSR, CSR, SSG

- Làm việc với Advanced Routing (Nested, Dynamic, Parallel)

---

2. Tech Stack

2.1. Core Framework

```
{
  "next": "^16.0.8",
  "react": "^19.0.0",
  "react-dom": "^19.0.0",
  "typescript": "^5.7.2"
}
```

2.2. Styling

```
{
  "tailwindcss": "^3.4.18",
  "autoprefixer": "^10.4.20",
  "postcss": "^8.4.49"
}
```

2.3. Form & Validation

```
{
  "react-hook-form": "^7.54.2",
  "zod": "^3.24.1",
  "@hookform/resolvers": "^3.9.1"
}
```

2.4. Utilities

```
{
  "date-fns": "^4.1.0",
  "react-hot-toast": "^2.4.0",
  "lucide-react": "^0.468.0",
  "clsx": "^2.1.1"
}
```

---

3. Cấu trúc thư mục

```
nextjs-blog/
├── app/
|   ├── (marketing)/
|   |   ├── layout.tsx
|   |   ├── page.tsx                # Homepage
|   |   ├── about/
|   |   |   └── page.tsx
|   |   └── contact/
|   |       └── page.tsx
```

```
|   |
|   ├── blog/
|   |   ├── page.tsx                # Blog listing
|   |   ├── loading.tsx
|   |   ├── error.tsx
|   |   ├── [slug]/
|   |   |   ├── page.tsx            # Post detail
|   |   |   ├── loading.tsx
|   |   |   └── @comments/
|   |   |       └── default.tsx     # Parallel route
|   |   └── category/
|   |       └── [category]/
|   |           └── page.tsx
|   |
|   ├── api/
|   |   ├── contact/
|   |   |   └── route.ts            # Contact form API
|   |   └── posts/
|   |       └── route.ts            # Posts API
|   |
|   ├── layout.tsx                  # Root layout
|   ├── loading.tsx
|   ├── error.tsx
|   ├── not-found.tsx
|   └── globals.css
|
├── components/
|   ├── layout/
|   |   ├── Header.tsx
|   |   ├── Footer.tsx
|   |   └── Navigation.tsx
|   |
|   ├── blog/
|   |   ├── PostCard.tsx            # Client component
|   |   ├── PostList.tsx            # Server component
|   |   ├── PostContent.tsx
|   |   ├── CategoryBadge.tsx
|   |   └── SearchBar.tsx           # Client component
|   |
|   ├── forms/
|   |   ├── ContactForm.tsx         # Client component
|   |   └── CommentForm.tsx         # Client component
|   |
|   └── ui/
|       ├── Button.tsx
|       ├── Input.tsx
|       ├── Textarea.tsx
|       └── Card.tsx
|
├── lib/
|   ├── api.ts                      # API functions
|   ├── utils.ts                    # Utility functions
|   ├── validations.ts              # Zod schemas
|   └── constants.ts
|
├── types/
```

```
│   └── index.ts                    # TypeScript types
│
├── data/
│   └── posts.json                  # Mock data
│
├── public/
│   ├── images/
│   │   ├── placeholder.jpg
│   │   └── posts/
│   └── favicon.ico
│
├── .env.local
├── next.config.js
├── tailwind.config.ts
├── tsconfig.json
└── package.json
```

---

4. Data Models

4.1. TypeScript Interfaces

```typescript
// types/index.ts

export interface Post {
  id: string
  title: string
  slug: string
  excerpt: string
  content: string
  coverImage: string
  author: Author
  category: Category
  publishedAt: string
  tags: string[]
  readingTime: number
}

export interface Author {
  id: string
  name: string
  avatar: string
  bio: string
  email?: string
}

export interface Category {
  id: string
  name: string
  slug: string
  description?: string
}

export interface Comment {
```

```
  id: string
  postId: string
  author: string
  email: string
  content: string
  createdAt: string
}

export interface ContactFormData {
  name: string
  email: string
  subject: string
  message: string
}
```

4.2. Mock Data Structure

```
// data/posts.json
[
  {
    "id": "1",
    "title": "Giới thiệu về Next.js 14",
    "slug": "gioi-thieu-nextjs-14",
    "excerpt": "Tìm hiểu về các tính năng mới trong Next.js 14 và App Router",
    "content": "# Nội dung bài viết...",
    "coverImage": "/images/posts/nextjs-14.jpg",
    "author": {
      "id": "1",
      "name": "Nguyễn Văn A",
      "avatar": "/images/authors/author-1.jpg",
      "bio": "Full-stack Developer"
    },
    "category": {
      "id": "1",
      "name": "Tutorial",
      "slug": "tutorial"
    },
    "publishedAt": "2024-01-15T10:00:00Z",
    "tags": ["nextjs", "react", "tutorial"],
    "readingTime": 5
  }
]
```

---

5. Routing Structure

5.1. Public Routes

Path	Description	Type
`/`	Homepage - Danh sách bài viết mới nhất	Server Component

Path	Description	Type
`/about`	Trang giới thiệu	Server Component
`/contact`	Form liên hệ	Client Component
`/blog`	Danh sách tất cả bài viết	Server Component
`/blog/[slug]`	Chi tiết bài viết	Server Component (Dynamic)
`/blog/category/[category]`	Bài viết theo danh mục	Server Component (Dynamic)

5.2. API Routes

Endpoint	Method	Description
`/api/contact`	POST	Xử lý form liên hệ
`/api/posts`	GET	Lấy danh sách bài viết
`/api/posts/[slug]`	GET	Lấy chi tiết bài viết

---

6. Component Specifications

6.1. Layout Components

Header Component

```
// components/layout/Header.tsx
'use client'

interface HeaderProps {
  // Props nếu cần
}

export default function Header({}: HeaderProps) {
  // Features:
  // - Logo
  // - Navigation menu
  // - Search bar (optional)
  // - Mobile menu toggle
}
```

Navigation Component

```
// components/layout/Navigation.tsx
'use client'

const navItems = [
  { label: 'Trang chủ', href: '/' },
```

```
  { label: 'Blog', href: '/blog' },
  { label: 'Giới thiệu', href: '/about' },
  { label: 'Liên hệ', href: '/contact' },
]

export default function Navigation() {
  // Use Next.js Link
  // Highlight active route
  // Responsive design
}
```

6.2. Blog Components

PostCard Component (Client Component)

```
// components/blog/PostCard.tsx
'use client'

import Image from 'next/image'
import Link from 'next/link'
import { Post } from '@/types'

interface PostCardProps {
  post: Post
}

export default function PostCard({ post }: PostCardProps) {
  // Features:
  // - Next.js Image optimization
  // - Link to post detail
  // - Category badge
  // - Reading time
  // - Excerpt truncation
  // - Hover effects
}
```

PostList Component (Server Component)

```
// components/blog/PostList.tsx
import { Post } from '@/types'
import PostCard from './PostCard'

interface PostListProps {
  posts: Post[]
}

export default async function PostList({ posts }: PostListProps) {
  // Server Component
  // Render grid of PostCard
  // Handle empty state
}
```

SearchBar Component (Client Component)

```
// components/blog/SearchBar.tsx
'use client'

import { useRouter, useSearchParams } from 'next/navigation'

export default function SearchBar() {
  // Features:
  // - Real-time search
  // - Update URL params
  // - Debounce input
  // - Clear button
}
```

6.3. Form Components

ContactForm Component

```
// components/forms/ContactForm.tsx
'use client'

import { useForm } from 'react-hook-form'
import { zodResolver } from '@hookform/resolvers/zod'
import { contactSchema } from '@/lib/validations'

export default function ContactForm() {
  // Features:
  // - React Hook Form
  // - Zod validation
  // - Error messages
  // - Submit to API route
  // - Success/error toast
  // - Loading state
  // - Reset form after submit
}
```

CommentForm Component

```
// components/forms/CommentForm.tsx
'use client'

interface CommentFormProps {
  postId: string
}

export default function CommentForm({ postId }: CommentFormProps) {
  // Similar to ContactForm
  // Simpler validation
  // Add comment to list (optimistic update)
}
```

7. Page Implementations

7.1. Homepage

```
// app/(marketing)/page.tsx
import { getPosts } from '@/lib/api'
import PostList from '@/components/blog/PostList'

export default async function HomePage() {
  const posts = await getPosts({ limit: 6 })

  return (
    <div>
      <section className="hero">
        {/* Hero section */}
      </section>

      <section className="latest-posts">
        <h2>Bài viết mới nhất</h2>
        <PostList posts={posts} />
      </section>
    </div>
  )
}
```

7.2. Blog Listing Page

```
// app/blog/page.tsx
import { getPosts, getCategories } from '@/lib/api'
import PostList from '@/components/blog/PostList'
import SearchBar from '@/components/blog/SearchBar'

interface BlogPageProps {
  searchParams: {
    q?: string
    category?: string
  }
}

export default async function BlogPage({ searchParams }: BlogPageProps) {
  const posts = await getPosts({
    search: searchParams.q,
    category: searchParams.category,
  })

  const categories = await getCategories()

  return (
    <div>
      <SearchBar />
      {/* Category filter */}
      <PostList posts={posts} />
      {/* Pagination */}
    </div>
```

```
  )
}
```

7.3. Blog Detail Page

```
// app/blog/[slug]/page.tsx
import { getPostBySlug, getAllPosts } from '@/lib/api'
import { notFound } from 'next/navigation'
import Image from 'next/image'
import PostContent from '@/components/blog/PostContent'

interface PostPageProps {
  params: {
    slug: string
  }
}

// Generate static params for SSG
export async function generateStaticParams() {
  const posts = await getAllPosts()
  return posts.map((post) => ({
    slug: post.slug,
  }))
}

// Generate metadata for SEO
export async function generateMetadata({ params }: PostPageProps) {
  const post = await getPostBySlug(params.slug)

  if (!post) return {}

  return {
    title: post.title,
    description: post.excerpt,
    openGraph: {
      title: post.title,
      description: post.excerpt,
      images: [post.coverImage],
    },
  }
}

export default async function PostPage({ params }: PostPageProps) {
  const post = await getPostBySlug(params.slug)

  if (!post) {
    notFound()
  }

  return (
    <article>
      <header>
        <h1>{post.title}</h1>
        <Image src={post.coverImage} alt={post.title} />
        {/* Author, date, reading time */}
```

```
      </header>

      <PostContent content={post.content} />

      {/* Comments section */}
    </article>
  )
}

// Revalidate every 1 hour
export const revalidate = 3600
```

7.4. Contact Page

```
// app/(marketing)/contact/page.tsx
import ContactForm from '@/components/forms/ContactForm'

export const metadata = {
  title: 'Liên hệ',
  description: 'Gửi tin nhắn cho chúng tôi',
}

export default function ContactPage() {
  return (
    <div className="container max-w-2xl">
      <h1>Liên hệ</h1>
      <p>Điền thông tin bên dưới để gửi tin nhắn cho chúng tôi.</p>
      <ContactForm />
    </div>
  )
}
```

---

8. API & Data Fetching

8.1. API Functions

```
// lib/api.ts
import { Post, Category } from '@/types'
import postsData from '@/data/posts.json'

export async function getPosts(options?: {
  limit?: number
  search?: string
  category?: string
}): Promise<Post[]> {
  // Simulate API delay
  await new Promise(resolve => setTimeout(resolve, 500))

  let posts = postsData as Post[]

  // Filter by search
  if (options?.search) {
```

```
    posts = posts.filter(post =>
      post.title.toLowerCase().includes(options.search!.toLowerCase())
    )
  }

  // Filter by category
  if (options?.category) {
    posts = posts.filter(post =>
      post.category.slug === options.category
    )
  }

  // Limit results
  if (options?.limit) {
    posts = posts.slice(0, options.limit)
  }

  return posts
}

export async function getPostBySlug(slug: string): Promise<Post | null> {
  await new Promise(resolve => setTimeout(resolve, 300))

  const post = postsData.find(p => p.slug === slug)
  return post || null
}

export async function getAllPosts(): Promise<Post[]> {
  return postsData as Post[]
}

export async function getCategories(): Promise<Category[]> {
  const posts = postsData as Post[]
  const categoryMap = new Map<string, Category>()

  posts.forEach(post => {
    categoryMap.set(post.category.id, post.category)
  })

  return Array.from(categoryMap.values())
}
```

8.2. Contact API Route

```
// app/api/contact/route.ts
import { NextRequest, NextResponse } from 'next/server'
import { z } from 'zod'

const contactSchema = z.object({
  name: z.string().min(2, 'Tên phải có ít nhất 2 ký tự'),
  email: z.string().email('Email không hợp lệ'),
  subject: z.string().min(5, 'Tiêu đề phải có ít nhất 5 ký tự'),
  message: z.string().min(10, 'Nội dung phải có ít nhất 10 ký tự'),
})
```

```
export async function POST(request: NextRequest) {
  try {
    const body = await request.json()

    // Validate
    const validatedData = contactSchema.parse(body)

    // TODO: Send email, save to database, etc.
    console.log('Contact form submission:', validatedData)

    // Simulate processing
    await new Promise(resolve => setTimeout(resolve, 1000))

    return NextResponse.json({
      success: true,
      message: 'Cảm ơn bạn đã liên hệ! Chúng tôi sẽ phản hồi sớm nhất có thể.',
    })
  } catch (error) {
    if (error instanceof z.ZodError) {
      return NextResponse.json(
        { success: false, errors: error.errors },
        { status: 400 }
      )
    }

    return NextResponse.json(
      { success: false, message: 'Đã có lỗi xảy ra' },
      { status: 500 }
    )
  }
}
```

9. Validation Schemas

```
// lib/validations.ts
import { z } from 'zod'

export const contactSchema = z.object({
  name: z.string()
    .min(2, 'Tên phải có ít nhất 2 ký tự')
    .max(50, 'Tên không được quá 50 ký tự'),
  email: z.string()
    .email('Email không hợp lệ'),
  subject: z.string()
    .min(5, 'Tiêu đề phải có ít nhất 5 ký tự')
    .max(100, 'Tiêu đề không được quá 100 ký tự'),
  message: z.string()
    .min(10, 'Nội dung phải có ít nhất 10 ký tự')
    .max(1000, 'Nội dung không được quá 1000 ký tự'),
})

export const commentSchema = z.object({
  author: z.string()
    .min(2, 'Tên phải có ít nhất 2 ký tự'),
```

```
  email: z.string()
    .email('Email không hợp lệ'),
  content: z.string()
    .min(5, 'Bình luận phải có ít nhất 5 ký tự')
    .max(500, 'Bình luận không được quá 500 ký tự'),
})
```

---

10. Styling Guidelines

10.1. Tailwind Configuration

```
// tailwind.config.ts
import type { Config } from 'tailwindcss'

const config: Config = {
  content: [
    './pages/**/*.{js,ts,jsx,tsx,mdx}',
    './components/**/*.{js,ts,jsx,tsx,mdx}',
    './app/**/*.{js,ts,jsx,tsx,mdx}',
  ],
  theme: {
    extend: {
      colors: {
        primary: {
          50: '#eff6ff',
          100: '#dbeafe',
          500: '#3b82f6',
          600: '#2563eb',
          700: '#1d4ed8',
        },
      },
      fontFamily: {
        sans: ['Inter', 'sans-serif'],
      },
    },
  },
  plugins: [],
}

export default config
```

10.2. Global Styles

```
/* app/globals.css */
@tailwind base;
@tailwind components;
@tailwind utilities;

@layer base {
  body {
    @apply bg-gray-50 text-gray-900;
  }
```

```
  h1 {
    @apply text-4xl font-bold mb-4;
  }

  h2 {
    @apply text-3xl font-semibold mb-3;
  }

  h3 {
    @apply text-2xl font-semibold mb-2;
  }
}

@layer components {
  .container {
    @apply max-w-7xl mx-auto px-4 sm:px-6 lg:px-8;
  }

  .btn {
    @apply px-4 py-2 rounded-md font-medium transition-colors;
  }

  .btn-primary {
    @apply bg-primary-600 text-white hover:bg-primary-700;
  }

  .card {
    @apply bg-white rounded-lg shadow-md p-6;
  }
}
```

---

11. Environment Variables

```
# .env.local
NEXT_PUBLIC_SITE_NAME="NextJS Blog"
NEXT_PUBLIC_SITE_URL="http://localhost:3000"
NEXT_PUBLIC_API_URL="http://localhost:3000/api"
```

---

12. Next.js Configuration

```
// next.config.js
/** @type {import('next').NextConfig} */
const nextConfig = {
  images: {
    domains: ['localhost'],
    formats: ['image/avif', 'image/webp'],
  },
  experimental: {
    // Enable if needed
  },
}

module.exports = nextConfig
```

---

13. Features Checklist

Phase 1: Basic Setup (Module 1)

- [ ] Khởi tạo Next.js project với TypeScript
- [ ] Cấu hình Tailwind CSS
- [ ] Tạo layout cơ bản (Header, Footer)
- [ ] Implement Homepage với mock data
- [ ] Sử dụng Next.js Image component
- [ ] CSS Modules hoặc Tailwind styling

Phase 2: Routing & Forms (Module 2)

- [ ] Setup App Router structure
- [ ] Tạo route groups: (marketing), blog
- [ ] Implement dynamic routes: [slug]
- [ ] Tạo Contact form với validation
- [ ] Phân biệt Client vs Server Components
- [ ] Static file serving (images, icons)
- [ ] Error handling & loading states

Phase 3: Advanced Features (Module 3)

- [ ] Data fetching từ API/mock data
- [ ] Implement SSR cho blog detail
- [ ] Implement CSR cho search/comments
- [ ] Generate static params cho SSG
- [ ] Nested routing: category/[category]
- [ ] Parallel routing: @comments slot

- [ ] SEO metadata động
- [ ] Revalidation strategy

---

14. Testing Requirements

14.1. Manual Testing Checklist

- [ ] Tất cả routes hoạt động đúng
- [ ] Form validation hiển thị lỗi chính xác
- [ ] Images load và optimize đúng
- [ ] Loading states hiển thị khi fetch data
- [ ] Error pages hiển thị khi cần
- [ ] Responsive trên mobile, tablet, desktop
- [ ] Navigation active state chính xác
- [ ] Search functionality hoạt động

14.2. Performance

- [ ] Lighthouse score > 90
- [ ] Images được optimize
- [ ] Code splitting hiệu quả
- [ ] No unnecessary client-side JS

---

15. Deployment

15.1. Build Commands

```
npm run build
npm run start
```

15.2. Deployment Platforms

- Vercel (Recommended)
- Netlify
- Cloudflare Pages

15.3. Pre-deployment Checklist

- [ ] Remove console.logs
- [ ] Update environment variables
- [ ] Test production build locally
- [ ] Verify all API routes work
- [ ] Check meta tags và SEO

---

16. Learning Outcomes Map

Module 1 → Implementation

- ✅ Giới thiệu Next.js → Project setup, folder structure
- ✅ Image component → Next/Image trong PostCard
- ✅ StyleSheet → Tailwind CSS, globals.css

Module 2 → Implementation

- ✅ App Router → Route groups, dynamic routes
- ✅ Client/Server Components → Forms (client), Posts (server)
- ✅ Form & Validation → ContactForm, CommentForm với Zod
- ✅ Static File Serving → /public folder

Module 3 → Implementation

- ✅ Data Fetching → API functions, fetch trong Server Components
- ✅ SSR → Blog detail page
- ✅ CSR → Search, Comments
- ✅ Advanced Routing → Nested, Dynamic, Parallel routes
- ✅ generateStaticParams → Blog posts SSG

---

17. Teaching Notes & Demo Flow

17.1. Module 1 - Cơ bản Next.js (4 giờ)

Demo Flow cho Giảng viên:

Buổi 1.1: Giới thiệu & Setup (1.5h)

```
1. Giới thiệu Next.js vs React (15 phút)
   → Show slide + so sánh code

2. Demo: Tạo project mới (30 phút)
   → Live code: npx create-next-app@latest
   → Giải thích cấu trúc thư mục
   → Show: app/, public/, components/

3. Demo: Cấu trúc App Router (30 phút)
   → Mở file: app/layout.tsx
   → Giải thích: Root Layout pattern
   → Mở file: app/page.tsx
   → Giải thích: Page component

4. Code walkthrough (15 phút)
   → Show ứng dụng demo đã build sẵn
   → Navigate qua các trang
   → Explain routing structure
```

Buổi 1.2: Image & Styling (1h)

```
1. Demo: Next.js Image Component (30 phút)
   → Mở: components/blog/PostCard.tsx
   → Highlight: import Image from 'next/image'
   → Explain: width, height, alt props
   → Show DevTools: Network tab (image optimization)
   → Compare: <img> vs <Image> performance

2. Demo: Tailwind CSS (30 phút)
   → Mở: app/globals.css
   → Explain: @tailwind directives
   → Mở: tailwind.config.ts
   → Show: Custom colors, fonts
   → Live edit: Thay đổi styling
```

Buổi 1.3: Hands-on Practice (1.5h)

```
Exercises cho học viên:

Exercise 1: Khởi tạo project (30 phút)
- Tạo Next.js project mới
- Customize layout với tên riêng
- Thêm logo vào Header

Exercise 2: Tạo PostCard component (45 phút)
- Clone component từ template
- Thêm Next/Image cho avatar
- Style với Tailwind
- Test responsive

Exercise 3: Tạo Homepage (15 phút)
- Import PostCard
- Render 3 posts với mock data
- Kiểm tra layout
```

Common Issues & Solutions:

```
// ❌ LỖI THƯỜNG GẶP 1: Image không load
<Image src="/logo.png" width={100} height={100} alt="Logo" />
// Lý do: File không có trong /public

// ✅ SOLUTION:
// Đảm bảo file ở: public/logo.png
// Hoặc dùng external URL và config next.config.js

// ❌ LỖI THƯỜNG GẶP 2: Tailwind không work
// Lý do: Chưa import globals.css

// ✅ SOLUTION:
// Thêm vào app/layout.tsx:
import './globals.css'
```

17.2. Module 2 - Routing & Forms (4 giờ)

Demo Flow cho Giảng viên:

Buổi 2.1: App Router Deep Dive (1.5h)

```
1. Demo: File-based Routing (45 phút)
   → Show: app/blog/page.tsx → /blog
   → Show: app/about/page.tsx → /about
   → Create live: app/test/page.tsx
   → Navigate browser → /test works!

   → Show: app/blog/[slug]/page.tsx
   → Explain: [slug] = dynamic segment
   → Test: /blog/hello → params.slug = "hello"

2. Demo: Server vs Client Components (45 phút)
   → Open: app/blog/page.tsx (Server Component)
   → Highlight: KHÔNG CÓ 'use client'
   → Can: async/await, direct DB calls
   → Cannot: useState, useEffect, onClick

   → Open: components/forms/ContactForm.tsx
   → Highlight: 'use client' ở đầu file
   → Can: hooks, event handlers
   → Explain: Interactive components cần Client
```

Teaching Points - Server vs Client:

```
// ============================================
// SERVER COMPONENT (Default)
// ============================================
// ✅ Có thể làm:
// - Fetch data trực tiếp
// - Access backend resources
// - Keep sensitive data (API keys)
// - Reduce bundle size

// ❌ KHÔNG thể làm:
// - useState, useEffect
// - onClick, onChange
// - Browser APIs (window, localStorage)

export default async function BlogPage() {
  const posts = await getPosts() // ✅ OK
  return <PostList posts={posts} />
}

// ============================================
// CLIENT COMPONENT ('use client')
// ============================================
// ✅ Có thể làm:
// - useState, useEffect
// - Event handlers
// - Browser APIs
```

```
// - Third-party interactive libraries

// ❌ Lưu ý:
// - Tăng bundle size
// - Không nên fetch sensitive data

'use client'

export default function SearchBar() {
  const [query, setQuery] = useState('') // ✅ OK

  const handleSearch = () => {
    // ✅ OK
  }

  return <input onChange={handleSearch} />
}
```

Buổi 2.2: Forms & Validation (1.5h)

```
1. Demo: React Hook Form (45 phút)
   → Open: components/forms/ContactForm.tsx
   → Explain: useForm hook
   → Explain: register, handleSubmit, errors
   → Live demo: Fill form → Show validation
   → Submit → Show console log

2. Demo: Zod Schema (45 phút)
   → Open: lib/validations.ts
   → Explain: Schema definition
   → Test live: Nhập email sai → Show error
   → Explain: zodResolver integration
```

Buổi 2.3: Practice (1h)

```
Exercise 1: Tạo Blog Routes (30 phút)
- Tạo app/blog/page.tsx
- Tạo app/blog/[slug]/page.tsx
- Test navigation

Exercise 2: Build Contact Form (30 phút)
- Copy template ContactForm.tsx
- Thêm validation cho phone field
- Test submit
```

Code Examples với Comments:

```
// ============================================
// EXAMPLE 1: Dynamic Route
// File: app/blog/[slug]/page.tsx
// ============================================

/**
 * TEACHING POINT: Dynamic Routing
```

```
 *
 * - [slug] trong tên folder = dynamic segment
 * - Next.js tự động pass params vào component
 * - Dùng cho: blog posts, product pages, user profiles
 */

interface PostPageProps {
  params: {
    slug: string  // ← This comes from [slug] folder name
  }
}

export default async function PostPage({ params }: PostPageProps) {
  // Access the dynamic segment
  console.log('Current slug:', params.slug)
  // URL: /blog/hello-world → params.slug = "hello-world"

  const post = await getPostBySlug(params.slug)

  return <article>{post.title}</article>
}

// ============================================
// EXAMPLE 2: Form với Validation
// File: components/forms/ContactForm.tsx
// ============================================

'use client' // ← REQUIRED for forms!

import { useForm } from 'react-hook-form'
import { zodResolver } from '@hookform/resolvers/zod'
import { contactSchema } from '@/lib/validations'

/**
 * TEACHING POINT: Form Handling
 *
 * 1. 'use client' - Forms cần interactivity
 * 2. useForm - React Hook Form hook
 * 3. zodResolver - Connect Zod với React Hook Form
 * 4. handleSubmit - Handle form submission
 */

export default function ContactForm() {
  const {
    register,    // ← Register input fields
    handleSubmit, // ← Handle form submit
    formState: { errors }, // ← Validation errors
  } = useForm({
    resolver: zodResolver(contactSchema), // ← Zod validation
  })

  const onSubmit = async (data: any) => {
    // TEACHING POINT: Submit to API route
    const response = await fetch('/api/contact', {
      method: 'POST',
      body: JSON.stringify(data),
```

```
    })
  }

  return (
    <form onSubmit={handleSubmit(onSubmit)}>
      {/* TEACHING POINT: Register input */}
      <input {...register('email')} />

      {/* TEACHING POINT: Show validation error */}
      {errors.email && (
        <span>{errors.email.message}</span>
      )}

      <button type="submit">Gửi</button>
    </form>
  )
}
```

17.3. Module 3 - Data Fetching & Advanced (4 giờ)

Demo Flow cho Giảng viên:

Buổi 3.1: Data Fetching Patterns (1.5h)

```
1. Demo: Server-side Fetching (45 phút)
   → Open: app/blog/[slug]/page.tsx
   → Explain: async function component
   → Explain: Direct fetch trong Server Component
   → Show: Data được render sẵn (View Source)
   → Explain: SEO benefits

2. Demo: Client-side Fetching (45 phút)
   → Open: components/blog/SearchBar.tsx
   → Explain: useState + useEffect pattern
   → Explain: Real-time search
   → Demo: Type → Filter ngay lập tức
```

Teaching Points - Data Fetching:

```
// ==============================================
// SERVER-SIDE FETCHING (SSR)
// ==============================================
/**
 * WHEN: Cần SEO, data thay đổi thường xuyên
 * WHERE: Server Components
 * HOW: async/await directly
 */

// File: app/blog/[slug]/page.tsx
export default async function PostPage({ params }: PostPageProps) {
  // ✅ Fetch on SERVER - mỗi request
  const post = await getPostBySlug(params.slug)
```

```
  // Data có sẵn khi page render
  // Google bot thấy được content
  return <article>{post.content}</article>
}

// ==============================================
// STATIC GENERATION (SSG)
// ==============================================
/**
 * WHEN: Content ít thay đổi, cần performance cao
 * WHERE: Server Components
 * HOW: generateStaticParams + revalidate
 */

// File: app/blog/[slug]/page.tsx

// TEACHING POINT: Generate pages at build time
export async function generateStaticParams() {
  const posts = await getAllPosts()

  // Generate static pages for these slugs
  return posts.map((post) => ({
    slug: post.slug,
  }))
}

// TEACHING POINT: Revalidate every 1 hour
export const revalidate = 3600

export default async function PostPage({ params }: PostPageProps) {
  const post = await getPostBySlug(params.slug)
  return <article>{post.content}</article>
}

// ==============================================
// CLIENT-SIDE FETCHING (CSR)
// ==============================================
/**
 * WHEN: Interactive features, real-time updates
 * WHERE: Client Components
 * HOW: useEffect + fetch
 */

'use client'

export default function SearchBar() {
  const [results, setResults] = useState([])

  useEffect(() => {
    // ✅ Fetch on CLIENT - after page load
    fetch('/api/search?q=' + query)
      .then(res => res.json())
      .then(data => setResults(data))
  }, [query])

  return <div>{/* Show results */}</div>
```

```
}
```

Buổi 3.2: Advanced Routing (1.5h)

```
1. Demo: Nested Routes (30 phút)
   → Show: app/blog/category/[category]/page.tsx
   → Explain: URL structure /blog/category/tutorial
   → Navigate: Test different categories

2. Demo: Parallel Routes (30 phút)
   → Show: app/blog/[slug]/@comments/page.tsx
   → Explain: @ prefix = slot
   → Explain: Multiple sections render parallel
   → Show: Layout with slots

3. Demo: Loading & Error States (30 phút)
   → Show: app/blog/loading.tsx
   → Explain: Automatic loading UI
   → Show: app/blog/error.tsx
   → Trigger error → Show error boundary
```

Buổi 3.3: Final Practice (1h)

```
Exercise 1: Implement SSR (30 phút)
- Fetch posts in blog/page.tsx
- Add loading.tsx
- Test with slow network

Exercise 2: Add Search (30 phút)
- Create SearchBar component
- Filter posts client-side
- Update URL params
```

Advanced Examples:

```
// ============================================
// EXAMPLE: Parallel Routes
// File: app/blog/[slug]/layout.tsx
// ============================================

/**
 * TEACHING POINT: Parallel Routing
 *
 * - @comments is a "slot"
 * - Renders parallel với children
 * - Useful cho: dashboards, modals, sidebars
 */

export default function PostLayout({
  children,  // ← Main content (page.tsx)
  comments,  // ← From @comments/page.tsx
}: {
  children: React.ReactNode
  comments: React.ReactNode
```

```
}) {
  return (
    <div className="grid grid-cols-3 gap-4">
      <div className="col-span-2">
        {children} {/* ← Blog post content */}
      </div>
      <div className="col-span-1">
        {comments} {/* ← Comments section */}
      </div>
    </div>
  )
}

// ============================================
// EXAMPLE: generateMetadata for SEO
// File: app/blog/[slug]/page.tsx
// ============================================

/**
 * TEACHING POINT: Dynamic SEO
 *
 * - generateMetadata runs on server
 * - Sets <title>, <meta> tags
 * - Important cho SEO
 */

export async function generateMetadata({ params }: PostPageProps) {
  const post = await getPostBySlug(params.slug)

  if (!post) return {}

  return {
    title: post.title,
    description: post.excerpt,
    openGraph: {
      title: post.title,
      description: post.excerpt,
      images: [post.coverImage],
    },
  }
}
```

---

18. Exercises & Solutions

Sau khi hoàn thành core features, có thể mở rộng:

1. **Authentication**
   - NextAuth.js
   - Protected routes
   - User profile
2. **Database Integration**
   - Prisma ORM

    - PostgreSQL/MySQL
    - CRUD operations
3. **Rich Text Editor**
    - TipTap hoặc Lexical
    - Markdown support
4. **Image Upload**
    - Cloudinary hoặc Uploadthing
    - Image optimization
5. **Comments System**
    - Real-time với Pusher
    - Comment moderation
6. **Analytics**
    - Google Analytics
    - View tracking
7. **PWA Features**
    - Service Worker
    - Offline support
8. **Internationalization**
    - next-intl
    - Multi-language support

---

18. Exercises & Solutions

18.1. Module 1 Exercises

Exercise 1.1: Khởi tạo Project ⭐

Thời gian: 30 phút
Mục tiêu: Làm quen với Next.js setup và cấu trúc cơ bản

Yêu cầu:

1. Tạo Next.js project mới với TypeScript
2. Cài đặt Tailwind CSS
3. Customize layout với thông tin cá nhân
4. Thay đổi favicon

Hướng dẫn:

```
# Bước 1: Tạo project
npx create-next-app@latest my-blog

# Options:
# ✅ TypeScript: Yes
# ✅ ESLint: Yes
```

```
# ✅ Tailwind CSS: Yes
# ✅ App Router: Yes
# ✅ Import alias: Yes (@/*)

# Bước 2: Chạy dev server
cd my-blog
npm run dev

# Bước 3: Mở http://localhost:3000
```

Tasks:

```
// TODO 1: Sửa app/layout.tsx
export const metadata: Metadata = {
  title: 'Blog của [Tên bạn]', // ← Đổi tên
  description: 'Blog cá nhân về lập trình',
}

// TODO 2: Thêm vào app/page.tsx
export default function Home() {
  return (
    <main className="container mx-auto p-8">
      <h1 className="text-4xl font-bold">
        Xin chào! Tôi là [Tên bạn]
      </h1>
      <p className="mt-4 text-gray-600">
        Đây là blog cá nhân của tôi
      </p>
    </main>
  )
}

// TODO 3: Thay favicon
// Thêm file mới: public/favicon.ico
```

Kiểm tra:

- [ ] Localhost:3000 hiển thị trang chủ
- [ ] Title hiển thị tên bạn
- [ ] Tailwind CSS hoạt động (text màu xanh)
- [ ] Favicon đổi thành favicon mới

Exercise 1.2: Tạo Header Component ⭐⭐

Thời gian: 45 phút
Mục tiêu: Làm việc với components và Next.js Image

Yêu cầu:

1. Tạo Header component
2. Sử dụng Next.js Image cho logo
3. Tạo navigation menu

4. Responsive design

Solution:

```
// components/layout/Header.tsx
'use client'

import Image from 'next/image'
import Link from 'next/link'

export default function Header() {
  return (
    <header className="bg-white shadow-sm">
      <div className="container mx-auto px-4 py-4">
        <div className="flex items-center justify-between">
          {/* Logo */}
          <Link href="/" className="flex items-center gap-2">
            <Image
              src="/logo.png"
              alt="Logo"
              width={40}
              height={40}
              priority
            />
            <span className="text-xl font-bold">My Blog</span>
          </Link>

          {/* Navigation */}
          <nav className="hidden md:flex gap-6">
            <Link href="/" className="hover:text-blue-600">
              Trang chủ
            </Link>
            <Link href="/blog" className="hover:text-blue-600">
              Blog
            </Link>
            <Link href="/about" className="hover:text-blue-600">
              Giới thiệu
            </Link>
            <Link href="/contact" className="hover:text-blue-600">
              Liên hệ
            </Link>
          </nav>
        </div>
      </div>
    </header>
  )
}
```

Kiểm tra:

- [ ] Header hiển thị trên mọi trang
- [ ] Logo load và optimize (check DevTools)
- [ ] Navigation links hoạt động
- [ ] Responsive (ẩn menu trên mobile)

Exercise 1.3: Tạo PostCard Component ⭐ ⭐

Thời gian: 45 phút
Mục tiêu: Làm việc với props và styling

Starter Code:

```
// types/index.ts
export interface Post {
  id: string
  title: string
  excerpt: string
  coverImage: string
  publishedAt: string
}
```

TODO: Hoàn thành component này

```
// components/blog/PostCard.tsx
'use client'

import Image from 'next/image'
import Link from 'next/link'
import { Post } from '@/types'

interface PostCardProps {
  post: Post
}

export default function PostCard({ post }: PostCardProps) {
  return (
    <Link href={`/blog/${post.id}`}>
      <article className="bg-white rounded-lg shadow-md overflow-hidden
hover:shadow-lg transition">
        {/* TODO 1: Thêm Image cho coverImage */}
        {/* Hint: width={400} height={250} */}

        <div className="p-4">
          {/* TODO 2: Hiển thị title */}
          {/* Hint: <h3 className="text-xl font-bold"> */}

          {/* TODO 3: Hiển thị excerpt */}
          {/* Hint: <p className="text-gray-600 mt-2"> */}

          {/* TODO 4: Hiển thị publishedAt */}
          {/* Hint: format date với date-fns */}
        </div>
      </article>
    </Link>
  )
}
```

Solution:

```
// components/blog/PostCard.tsx
'use client'

import Image from 'next/image'
import Link from 'next/link'
import { format } from 'date-fns'
import { vi } from 'date-fns/locale'
import { Post } from '@/types'

interface PostCardProps {
  post: Post
}

export default function PostCard({ post }: PostCardProps) {
  return (
    <Link href={`/blog/${post.id}`}>
      <article className="bg-white rounded-lg shadow-md overflow-hidden
hover:shadow-lg transition-shadow">
        {/* Cover Image */}
        <div className="relative h-48">
          <Image
            src={post.coverImage}
            alt={post.title}
            fill
            className="object-cover"
          />
        </div>

        <div className="p-4">
          {/* Title */}
          <h3 className="text-xl font-bold mb-2 line-clamp-2">
            {post.title}
          </h3>

          {/* Excerpt */}
          <p className="text-gray-600 text-sm mb-4 line-clamp-3">
            {post.excerpt}
          </p>

          {/* Date */}
          <time className="text-xs text-gray-400">
            {format(new Date(post.publishedAt), 'dd MMM yyyy', {
              locale: vi,
            })}
          </time>
        </div>
      </article>
    </Link>
  )
}
```

18.2. Module 2 Exercises

Exercise 2.1: Tạo Blog Routes ⭐⭐

Thời gian: 30 phút
Mục tiêu: Làm quen với App Router

Yêu cầu:

1. Tạo route `/blog` - hiển thị danh sách posts
2. Tạo route `/blog/[slug]` - hiển thị chi tiết post
3. Test navigation giữa các routes

TODO:

```
# Tạo các file sau:
app/
  blog/
    page.tsx          # ← TODO: Create this
    [slug]/
      page.tsx        # ← TODO: Create this
```

Solution:

```
// app/blog/page.tsx
import PostCard from '@/components/blog/PostCard'

// Mock data
const posts = [
  {
    id: '1',
    title: 'Bài viết đầu tiên',
    excerpt: 'Đây là excerpt...',
    coverImage: '/images/post-1.jpg',
    publishedAt: '2024-01-01',
  },
  // ... more posts
]

export default function BlogPage() {
  return (
    <div className="container mx-auto px-4 py-8">
      <h1 className="text-4xl font-bold mb-8">Blog</h1>

      <div className="grid md:grid-cols-2 lg:grid-cols-3 gap-6">
        {posts.map((post) => (
          <PostCard key={post.id} post={post} />
        ))}
      </div>
    </div>
  )
}
```

```
// app/blog/[slug]/page.tsx
```

```
interface PostPageProps {
  params: {
    slug: string
  }
}

export default function PostPage({ params }: PostPageProps) {
  return (
    <div className="container mx-auto px-4 py-8">
      <h1 className="text-4xl font-bold">
        Bài viết: {params.slug}
      </h1>
      <p className="mt-4">
        Nội dung bài viết sẽ hiển thị ở đây...
      </p>
    </div>
  )
}
```

Test:

- Navigate to `/blog` → Thấy danh sách
- Click vào post → Navigate to `/blog/1`
- URL thay đổi đúng

Exercise 2.2: Contact Form với Validation ⭐⭐⭐

Thời gian: 1 giờ
Mục tiêu: Form handling và validation

Setup:

```
npm install react-hook-form zod @hookform/resolvers
```

TODO: Hoàn thành form này

```
// lib/validations.ts
import { z } from 'zod'

export const contactSchema = z.object({
  // TODO 1: Thêm validation cho name (min 2 chars)
  // TODO 2: Thêm validation cho email
  // TODO 3: Thêm validation cho message (min 10 chars)
})

// components/forms/ContactForm.tsx
'use client'

import { useForm } from 'react-hook-form'
import { zodResolver } from '@hookform/resolvers/zod'
import { contactSchema } from '@/lib/validations'

export default function ContactForm() {
```

```
  const {
    register,
    handleSubmit,
    formState: { errors },
  } = useForm({
    resolver: zodResolver(contactSchema),
  })

  const onSubmit = async (data: any) => {
    // TODO 4: Log data ra console
    // TODO 5: Show alert thành công
  }

  return (
    <form onSubmit={handleSubmit(onSubmit)} className="space-y-4">
      {/* TODO 6: Tạo input cho name */}
      {/* TODO 7: Hiển thị error message */}

      {/* TODO 8: Tạo input cho email */}
      {/* TODO 9: Hiển thị error message */}

      {/* TODO 10: Tạo textarea cho message */}
      {/* TODO 11: Hiển thị error message */}

      <button type="submit">Gửi</button>
    </form>
  )
}
```

Solution:

```
// lib/validations.ts
import { z } from 'zod'

export const contactSchema = z.object({
  name: z.string()
    .min(2, 'Tên phải có ít nhất 2 ký tự')
    .max(50, 'Tên không được quá 50 ký tự'),
  email: z.string()
    .email('Email không hợp lệ'),
  message: z.string()
    .min(10, 'Tin nhắn phải có ít nhất 10 ký tự')
    .max(500, 'Tin nhắn không được quá 500 ký tự'),
})

export type ContactFormData = z.infer<typeof contactSchema>
```

```
// components/forms/ContactForm.tsx
'use client'

import { useForm } from 'react-hook-form'
import { zodResolver } from '@hookform/resolvers/zod'
import { contactSchema, ContactFormData } from '@/lib/validations'
```

```
export default function ContactForm() {
  const {
    register,
    handleSubmit,
    formState: { errors, isSubmitting },
    reset,
  } = useForm<ContactFormData>({
    resolver: zodResolver(contactSchema),
  })

  const onSubmit = async (data: ContactFormData) => {
    console.log('Form data:', data)
    alert('Gửi thành công!')
    reset()
  }

  return (
    <form onSubmit={handleSubmit(onSubmit)} className="max-w-md space-y-4">
      {/* Name Field */}
      <div>
        <label className="block text-sm font-medium mb-1">
          Họ tên
        </label>
        <input
          {...register('name')}
          className="w-full px-4 py-2 border rounded-md"
          placeholder="Nguyễn Văn A"
        />
        {errors.name && (
          <p className="text-red-500 text-sm mt-1">
            {errors.name.message}
          </p>
        )}
      </div>

      {/* Email Field */}
      <div>
        <label className="block text-sm font-medium mb-1">
          Email
        </label>
        <input
          {...register('email')}
          type="email"
          className="w-full px-4 py-2 border rounded-md"
          placeholder="example@email.com"
        />
        {errors.email && (
          <p className="text-red-500 text-sm mt-1">
            {errors.email.message}
          </p>
        )}
      </div>

      {/* Message Field */}
      <div>
        <label className="block text-sm font-medium mb-1">
```

```
          Tin nhắn
        </label>
        <textarea
          {...register('message')}
          rows={4}
          className="w-full px-4 py-2 border rounded-md"
          placeholder="Nội dung tin nhắn..."
        />
        {errors.message && (
          <p className="text-red-500 text-sm mt-1">
            {errors.message.message}
          </p>
        )}
      </div>

      {/* Submit Button */}
      <button
        type="submit"
        disabled={isSubmitting}
        className="w-full bg-blue-600 text-white py-2 rounded-md hover:bg-blue-
700 disabled:bg-gray-400"
      >
        {isSubmitting ? 'Đang gửi...' : 'Gửi tin nhắn'}
      </button>
    </form>
  )
}
```

Test Cases:

- [ ] Submit form trống → Show validation errors
- [ ] Nhập email sai format → Show error
- [ ] Nhập tin nhắn < 10 ký tự → Show error
- [ ] Fill đúng → Submit thành công
- [ ] Form reset sau khi submit

18.3. Module 3 Exercises

Exercise 3.1: Data Fetching với SSR ⭐⭐⭐

Thời gian: 45 phút
Mục tiêu: Fetch data trong Server Component

Yêu cầu:

1. Tạo API function để fetch posts
2. Sử dụng trong Server Component
3. Thêm loading state

TODO:

```typescript
// lib/api.ts
import { Post } from '@/types'

export async function getPosts(): Promise<Post[]> {
  // TODO 1: Fetch từ JSONPlaceholder
  // URL: https://jsonplaceholder.typicode.com/posts
  // TODO 2: Transform data thành Post interface
  // TODO 3: Return mảng posts
}
```

Solution:

```typescript
// lib/api.ts
import { Post } from '@/types'

export async function getPosts(): Promise<Post[]> {
  try {
    const response = await fetch(
      'https://jsonplaceholder.typicode.com/posts',
      {
        next: { revalidate: 3600 }, // Cache 1 hour
      }
    )

    if (!response.ok) {
      throw new Error('Failed to fetch posts')
    }

    const data = await response.json()

    // Transform to Post interface
    const posts: Post[] = data.slice(0, 10).map((item: any) => ({
      id: item.id.toString(),
      title: item.title,
      slug: item.title.toLowerCase().replace(/ /g, '-'),
      excerpt: item.body.substring(0, 100) + '...',
      content: item.body,
      coverImage: `https://picsum.photos/400/250?random=${item.id}`,
      author: {
        id: '1',
        name: 'Admin',
        avatar: '/images/avatar.jpg',
        bio: 'Blog author',
      },
      category: {
        id: '1',
        name: 'General',
        slug: 'general',
      },
      publishedAt: new Date().toISOString(),
      tags: [],
      readingTime: 5,
    }))

    return posts
```

```
  } catch (error) {
    console.error('Error fetching posts:', error)
    return []
  }
}

export async function getPostBySlug(slug: string): Promise<Post | null> {
  const posts = await getPosts()
  return posts.find((post) => post.slug === slug) || null
}
```

```
// app/blog/page.tsx
import { getPosts } from '@/lib/api'
import PostCard from '@/components/blog/PostCard'

export default async function BlogPage() {
  // Fetch on server
  const posts = await getPosts()

  return (
    <div className="container mx-auto px-4 py-8">
      <h1 className="text-4xl font-bold mb-8">Blog</h1>

      <div className="grid md:grid-cols-2 lg:grid-cols-3 gap-6">
        {posts.map((post) => (
          <PostCard key={post.id} post={post} />
        ))}
      </div>
    </div>
  )
}
```

```
// app/blog/loading.tsx
export default function Loading() {
  return (
    <div className="container mx-auto px-4 py-8">
      <div className="animate-pulse space-y-4">
        <div className="h-8 bg-gray-200 rounded w-1/4"></div>
        <div className="grid md:grid-cols-3 gap-6">
          {[1, 2, 3].map((i) => (
            <div key={i} className="bg-gray-200 h-64 rounded"></div>
          ))}
        </div>
      </div>
    </div>
  )
}
```

Test:

- [ ] Navigate to /blog → Show loading
- [ ] Posts load từ API
- [ ] Data cached (reload nhanh hơn)

☐ View page source → See rendered HTML

Exercise 3.2: Client-side Search ⭐⭐⭐

Thời gian: 45 phút
Mục tiêu: Interactive features với Client Component

TODO:

```
// components/blog/SearchBar.tsx
'use client'

import { useRouter, useSearchParams } from 'next/navigation'
import { useState, useEffect } from 'react'

export default function SearchBar() {
  // TODO 1: Get current search from URL
  // TODO 2: Create state for input value
  // TODO 3: Update URL when search changes
  // TODO 4: Debounce search input
}
```

Solution:

```
// components/blog/SearchBar.tsx
'use client'

import { useRouter, useSearchParams } from 'next/navigation'
import { useState, useEffect } from 'react'
import { useDebounce } from '@/hooks/useDebounce'

export default function SearchBar() {
  const router = useRouter()
  const searchParams = useSearchParams()
  const [query, setQuery] = useState(searchParams.get('q') || '')

  // Debounce search
  const debouncedQuery = useDebounce(query, 300)

  useEffect(() => {
    // Update URL params
    const params = new URLSearchParams(searchParams.toString())

    if (debouncedQuery) {
      params.set('q', debouncedQuery)
    } else {
      params.delete('q')
    }

    router.push(`?${params.toString()}`)
  }, [debouncedQuery, router, searchParams])

  return (
    <div className="mb-6">
```

```
      <input
        type="search"
        value={query}
        onChange={(e) => setQuery(e.target.value)}
        placeholder="Tìm kiếm bài viết..."
        className="w-full px-4 py-2 border rounded-md"
      />
    </div>
  )
}
```

```
// hooks/useDebounce.ts
import { useState, useEffect } from 'react'

export function useDebounce<T>(value: T, delay: number): T {
  const [debouncedValue, setDebouncedValue] = useState(value)

  useEffect(() => {
    const timer = setTimeout(() => {
      setDebouncedValue(value)
    }, delay)

    return () => {
      clearTimeout(timer)
    }
  }, [value, delay])

  return debouncedValue
}
```

---

19. Common Errors & Troubleshooting

Issue 1: "use client" directive

Problem: Server Component errors khi sử dụng hooks
Solution: Thêm `'use client'` ở đầu file component

Issue 2: Image optimization errors

Problem: Images từ external sources không load
Solution: Thêm domain vào `next.config.js` → `images.domains`

Issue 3: Hydration errors

Problem: Mismatch giữa server và client render
Solution: Đảm bảo HTML giống nhau, tránh `window`, `localStorage` trong Server Components

Issue 4: Dynamic route not found

Problem: 404 khi access dynamic route
Solution: Kiểm tra folder structure, đảm bảo `[slug]` syntax đúng

---

19. Common Errors & Troubleshooting

19.1. Setup & Configuration Errors

Error 1: "You're importing a component that needs useState..."

```
Error: You're importing a component that needs useState.
It only works in a Client Component but none of its parents are marked with "use
client"
```

Nguyên nhân:

- Sử dụng hooks (useState, useEffect) trong Server Component
- Sử dụng event handlers (onClick, onChange) mà không có 'use client'

Solution:

```
// ❌ WRONG
export default function MyComponent() {
  const [count, setCount] = useState(0) // Error!
  return <button onClick={() => setCount(count + 1)}>Click</button>
}

// ✅ CORRECT
'use client' // ← Thêm directive này

export default function MyComponent() {
  const [count, setCount] = useState(0) // OK!
  return <button onClick={() => setCount(count + 1)}>Click</button>
}
```

Error 2: Image Error - Invalid src prop

```
Error: Invalid src prop on `next/image`, hostname "example.com" is not configured
```

Nguyên nhân:

- External image URL chưa được config trong next.config.js

Solution:

```
// next.config.js
/** @type {import('next').NextConfig} */
const nextConfig = {
  images: {
    remotePatterns: [
```

```
      {
        protocol: 'https',
        hostname: 'picsum.photos',
      },
      {
        protocol: 'https',
        hostname: '**.example.com', // Wildcard subdomain
      },
    ],
  },
}

module.exports = nextConfig
```

Error 3: Module not found - Can't resolve '@/...'

```
Error: Module not found: Can't resolve '@/components/...'
```

Nguyên nhân:

- Path alias chưa được config đúng

Solution:

```
// tsconfig.json
{
  "compilerOptions": {
    "baseUrl": ".",
    "paths": {
      "@/*": ["./*"]  // ← Đảm bảo có config này
    }
  }
}
```

19.2. Routing Errors

Error 4: 404 - Page Not Found

```
404 | This page could not be found.
```

Checklist:

- [ ] File có đúng tên `page.tsx` không? (không phải `Page.tsx` hay `index.tsx`)
- [ ] Folder structure đúng chưa?
- [ ] Component có export default không?

Example:

```
// ❌ WRONG - Sai tên file
app/blog/index.tsx  // Should be page.tsx

// ❌ WRONG - Thiếu export default
export function BlogPage() { ... }

// ✅ CORRECT
// File: app/blog/page.tsx
export default function BlogPage() { ... }
```

Error 5: Dynamic Route không hoạt động

```
Params is undefined trong dynamic route
```

Solution:

```
// ❌ WRONG - Thiếu await params
export default function PostPage({ params }) {
  console.log(params.slug) // Undefined!
}

// ✅ CORRECT - Next.js 15+
export default async function PostPage({
  params
}: {
  params: Promise<{ slug: string }>
}) {
  const { slug } = await params
  console.log(slug) // Works!
}
```

19.3. Data Fetching Errors

Error 6: Fetch Failed - CORS Error

```
Error: Failed to fetch
Access to fetch has been blocked by CORS policy
```

Nguyên nhân:

- Fetch từ Client Component đến API không hỗ trợ CORS

Solution 1: Sử dụng Server Component

```
// ✅ Fetch trong Server Component (không có CORS issue)
// app/blog/page.tsx
export default async function BlogPage() {
  const res = await fetch('https://api.example.com/posts')
  const data = await res.json()
  return <div>{/* render data */}</div>
}
```

Solution 2: Dùng Next.js API Route làm proxy

```
// app/api/posts/route.ts
export async function GET() {
  const res = await fetch('https://api.example.com/posts')
  const data = await res.json()
  return Response.json(data)
}

// Client Component
'use client'
export default function MyComponent() {
  const res = await fetch('/api/posts') // ← Internal API, no CORS
  const data = await res.json()
}
```

Error 7: Hydration Mismatch

```
Error: Hydration failed because the initial UI does not match what was rendered
on the server
```

Nguyên nhân:

- HTML khác nhau giữa server và client
- Sử dụng `window`, `localStorage` trong render

Common Mistakes:

```
// ❌ WRONG
export default function MyComponent() {
  return (
    <div>
      {typeof window !== 'undefined' ? 'Client' : 'Server'}
    </div>
  )
}

// ❌ WRONG
export default function MyComponent() {
  const [mounted, setMounted] = useState(false)

  useEffect(() => {
    setMounted(true)
  }, [])

  return <div>{mounted ? <ClientOnly /> : null}</div>
}
```

✅ **CORRECT Solutions:**

```
// Solution 1: Use 'use client' + useEffect
'use client'
```

```
export default function MyComponent() {
  const [clientData, setClientData] = useState(null)

  useEffect(() => {
    // Chỉ chạy trên client
    setClientData(window.localStorage.getItem('key'))
  }, [])

  return <div>{clientData}</div>
}

// Solution 2: Dynamic import with ssr: false
import dynamic from 'next/dynamic'

const ClientOnlyComponent = dynamic(
  () => import('./ClientOnlyComponent'),
  { ssr: false }
)

export default function MyComponent() {
  return <ClientOnlyComponent />
}
```

19.4. Form & Validation Errors

Error 8: React Hook Form - useForm is not a function

```
TypeError: useForm is not a function
```

Nguyên nhân:

- Thiếu 'use client' directive

Solution:

```
// ❌ WRONG
import { useForm } from 'react-hook-form'

export default function ContactForm() {
  const { register } = useForm() // Error!
}

// ✅ CORRECT
'use client' // ← Thêm directive này

import { useForm } from 'react-hook-form'

export default function ContactForm() {
  const { register } = useForm() // Works!
}
```

Error 9: Zod Validation không hiển thị message

```
Validation fails nhưng không có error message
```

Solution:

```
// ❌ WRONG - Thiếu error message trong schema
const schema = z.object({
  email: z.string().email(), // No custom message
})

// ✅ CORRECT - Có custom message
const schema = z.object({
  email: z.string()
    .min(1, 'Email là bắt buộc')
    .email('Email không hợp lệ'),
  name: z.string()
    .min(2, 'Tên phải có ít nhất 2 ký tự'),
})
```

19.5. Build & Deployment Errors

Error 10: Build Failed - Type Error

```
Type error: Property 'slug' does not exist on type 'never'
```

Nguyên nhân:

- TypeScript types không chính xác

Solution:

```
// ❌ WRONG
export default async function PostPage({ params }) {
  const { slug } = params // Type error!
}

// ✅ CORRECT - Proper typing
interface PostPageProps {
  params: {
    slug: string
  }
}

export default async function PostPage({ params }: PostPageProps) {
  const { slug } = params // Works!
}
```

Error 11: Production Build - Image Optimization Error

```
Error: Unable to optimize image
```

Nguyên nhân:

- Image quá lớn hoặc format không hỗ trợ

Solution:

```
// next.config.js
const nextConfig = {
  images: {
    // Increase limit
    deviceSizes: [640, 750, 828, 1080, 1200, 1920],
    imageSizes: [16, 32, 48, 64, 96, 128, 256, 384],

    // Or disable optimization for certain images
    unoptimized: process.env.NODE_ENV === 'development',
  },
}
```

19.6. Performance Issues

Issue 1: Slow Page Load

Symptoms:

- Trang load chậm
- Nhiều requests

Debug Steps:

```
# 1. Check bundle size
npm run build
# Look for large chunks

# 2. Analyze bundle
npm install -D @next/bundle-analyzer

# next.config.js
const withBundleAnalyzer = require('@next/bundle-analyzer')({
  enabled: process.env.ANALYZE === 'true',
})

module.exports = withBundleAnalyzer(nextConfig)

# Run analysis
ANALYZE=true npm run build
```

Solutions:

- Sử dụng dynamic imports

- Lazy load components
- Optimize images
- Implement pagination

Issue 2: Too Many Re-renders

Symptoms:

- Console warning: "Maximum update depth exceeded"
- Page freezes

Common Cause:

```
// ❌ WRONG - Infinite loop
export default function MyComponent() {
  const [count, setCount] = useState(0)

  setCount(count + 1) // Called every render!

  return <div>{count}</div>
}
```

Solution:

```
// ✅ CORRECT - Use useEffect
export default function MyComponent() {
  const [count, setCount] = useState(0)

  useEffect(() => {
    setCount(count + 1)
  }, []) // Only run once

  return <div>{count}</div>
}
```

19.7. Quick Reference - Debug Checklist

Khi gặp lỗi, check theo thứ tự:

1. ✅ **Console Errors**
   - Mở DevTools Console
   - Đọc error message đầu tiên
   - Check stack trace
2. ✅ **File Structure**
   - Tên file đúng chưa? (`page.tsx`, `layout.tsx`)
   - Folder structure đúng chưa?
   - Export default chưa?
3. ✅ **'use client' Directive**

    - Component có dùng hooks không? → Cần 'use client'
    - Component có event handlers không? → Cần 'use client'
4. ✅ **TypeScript Types**
    - Props có đúng type không?
    - Params có await không? (Next.js 15+)
5. ✅ **Dependencies**
    - `npm install` đã chạy chưa?
    - Dependencies đúng version chưa?
    - Restart dev server
6. ✅ **Cache Issues**

    ```
    # Clear Next.js cache
    rm -rf .next
    npm run dev
    ```

---

20. Extended Features (Optional)

Documentation

- Next.js Docs
- React Hook Form
- Zod
- Tailwind CSS

Tutorials

- Next.js Learn
- TypeScript Handbook

---

20. Implementation Priority

Must Have (P0)

1. Basic routing structure
2. Homepage với post listing
3. Blog detail page
4. Contact form với validation
5. Responsive layout

Should Have (P1)

6. Search functionality
7. Category filtering
8. Loading & error states
9. SEO metadata
10. Comments section

Nice to Have (P2)

11. Dark mode
12. Social sharing
13. Related posts
14. Reading progress bar
15. Table of contents

---

21. Git Workflow for Teaching

21.1. Commit Strategy (Progressive Learning)

Để học viên dễ follow, tạo commits theo từng feature:

```
# Module 1 Commits
git commit -m "feat: initial Next.js setup with TypeScript"
git commit -m "feat: add Tailwind CSS configuration"
git commit -m "feat: create basic layout with Header and Footer"
git commit -m "feat: add homepage with hero section"
git commit -m "feat: create PostCard component with Next/Image"
git commit -m "feat: add mock data for posts"

# Module 2 Commits
git commit -m "feat: setup App Router structure"
git commit -m "feat: create blog listing page"
git commit -m "feat: add dynamic route for blog detail"
git commit -m "feat: implement ContactForm with React Hook Form"
git commit -m "feat: add Zod validation for contact form"
git commit -m "feat: create About page"

# Module 3 Commits
git commit -m "feat: add API functions for data fetching"
git commit -m "feat: implement SSR for blog detail page"
git commit -m "feat: add loading states"
git commit -m "feat: implement SearchBar with CSR"
git commit -m "feat: add category filtering"
git commit -m "feat: implement generateStaticParams for SSG"
git commit -m "feat: add SEO metadata"

# Polish Commits
git commit -m "style: improve responsive design"
git commit -m "feat: add error boundaries"
```

```
git commit -m "docs: add README and comments"
```

21.2. Branch Strategy

```
# Main branch - Hoàn chỉnh
main

# Feature branches - Từng module
├── module-1-basics
├── module-2-routing
└── module-3-advanced

# Student branch - Starter template
└── starter (minimal setup, exercise files)
```

Hướng dẫn cho học viên:

```
# Clone repo
git clone <repo-url>

# Checkout starter branch
git checkout starter

# Follow commits trong main để xem solution
git checkout main
git log --oneline
git show <commit-hash>
```

---

22. Teaching Resources & Materials

22.1. Slide Deck Outline

Module 1: Giới thiệu Next.js

```
Slide 1: What is Next.js?
- React framework for production
- Built-in features: Routing, SSR, Image optimization
- Vercel support

Slide 2: Next.js vs React
[Table comparison]

Slide 3: App Router Architecture
[Diagram: folder structure]

Slide 4: Server vs Client Components
[Visual comparison]

Slide 5: Next.js Image Component
[Before/After performance comparison]
```

Module 2: Routing & Forms

```
Slide 6: File-based Routing
[Diagram: folder → URL]

Slide 7: Dynamic Routes
[Examples: [slug], [...slug], [[...slug]]]

Slide 8: Form Handling Best Practices
[Code comparison]

Slide 9: React Hook Form + Zod
[Integration diagram]
```

Module 3: Data Fetching

```
Slide 10: SSR vs CSR vs SSG
[Comparison table + use cases]

Slide 11: Data Fetching Patterns
[Code examples]

Slide 12: Advanced Routing
[Examples: Parallel, Intercepting]
```

22.2. Demo Scenarios

Scenario 1: Show Image Optimization (5 phút)

```
1. Mở Network tab trong DevTools
2. Load page với <img> tag
   → Show: Original size (e.g., 2MB)

3. Thay bằng <Image> component
   → Show: Optimized size (e.g., 100KB)
   → Show: WebP format
   → Show: Responsive srcset
```

Scenario 2: Demonstrate SSR (5 phút)

```
1. Mở blog detail page
2. View Page Source
   → Show: HTML content có sẵn

3. Disable JavaScript
   → Show: Content vẫn hiển thị

4. So sánh với CSR page
   → Show: No content without JS
```

Scenario 3: Form Validation Demo (10 phút)

```
1. Submit empty form
   → Show: Validation errors

2. Nhập email sai format
   → Show: Email error

3. Fill đúng → Submit
   → Show: Success message

4. Open code
   → Explain: Zod schema
   → Explain: Error handling
```

22.3. Code Snippet Library

Tạo file snippets để học viên copy nhanh:

snippets/header.tsx

```
// Basic Header component
'use client'

import Link from 'next/link'

export default function Header() {
  return (
    <header className="bg-white shadow">
      <nav className="container mx-auto px-4 py-4">
        <Link href="/">Home</Link>
        {/* Add more links */}
      </nav>
    </header>
  )
}
```

snippets/post-card.tsx

```
// Basic PostCard component
'use client'

import Image from 'next/image'
import Link from 'next/link'

export default function PostCard({ post }) {
  return (
    <Link href={`/blog/${post.slug}`}>
      <article className="bg-white rounded-lg shadow">
        <Image
          src={post.coverImage}
          alt={post.title}
          width={400}
          height={250}
        />
        <div className="p-4">
```

```
          <h3>{post.title}</h3>
          <p>{post.excerpt}</p>
        </div>
      </article>
    </Link>
  )
}
```

22.4. Quick Reference Cards

Card 1: Server vs Client Component

```
┌──────────────────────────────────────┐
│ SERVER COMPONENT (Default)           │
├──────────────────────────────────────┤
│ ✅ async/await                       │
│ ✅ Direct DB access                  │
│ ✅ Sensitive data                    │
│ ❌ useState/useEffect                │
│ ❌ onClick/onChange                  │
└──────────────────────────────────────┘

┌──────────────────────────────────────┐
│ CLIENT COMPONENT ('use client')      │
├──────────────────────────────────────┤
│ ✅ useState/useEffect                │
│ ✅ Event handlers                    │
│ ✅ Browser APIs                      │
│ ❌ Direct DB access                  │
│ ❌ Sensitive data                    │
└──────────────────────────────────────┘
```

Card 2: Routing Cheat Sheet

```
┌──────────────────────────────────────┐
│ FILE STRUCTURE → URL                 │
├──────────────────────────────────────┤
│ app/page.tsx           → /           │
│ app/about/page.tsx     → /about      │
│ app/blog/page.tsx      → /blog       │
│ app/blog/[slug]/page.tsx → /blog/:slug  │
│ app/blog/[...slug]/page.tsx → /blog/*   │
└──────────────────────────────────────┘
```

Card 3: Data Fetching Methods

```
┌──────────────────────────────────────┐
│ SSR (Server-Side Rendering)          │
├──────────────────────────────────────┤
│ When: Dynamic data, mỗi request      │
│ How: async component + fetch         │
│ SEO: ✅ Excellent                    │
└──────────────────────────────────────┘
```

```
┌──────────────────────────────────────┐
│ SSG (Static Site Generation)         │
├──────────────────────────────────────┤
│ When: Static content, build time     │
│ How: generateStaticParams            │
│ Performance: ✅ Fastest              │
└──────────────────────────────────────┘


┌──────────────────────────────────────┐
│ CSR (Client-Side Rendering)          │
├──────────────────────────────────────┤
│ When: Interactive, user-specific     │
│ How: useEffect + fetch               │
│ SEO: ❌ Limited                      │
└──────────────────────────────────────┘
```

22.5. Assessment Checklist

Quiz Questions:

Module 1:

- [ ] Next.js là gì? Khác gì với React?
- [ ] Giải thích cấu trúc folder trong App Router
- [ ] Next/Image có lợi ích gì so với ?
- [ ] Làm thế nào để add custom fonts?

Module 2:

- [ ] Tạo dynamic route như thế nào?
- [ ] Server Component khác Client Component ở điểm nào?
- [ ] Khi nào cần 'use client'?
- [ ] React Hook Form hoạt động ra sao?

Module 3:

- [ ] SSR vs CSR vs SSG - khác nhau thế nào?
- [ ] generateStaticParams dùng để làm gì?
- [ ] Cách implement search functionality?
- [ ] Parallel routing hoạt động như thế nào?

Practical Test:

```
Bài kiểm tra thực hành (60 phút):

1. Tạo route mới /products với:
   - Product listing page
   - Product detail page [id]
   - Search functionality

2. Requirements:
   - Fetch data từ fake API
   - Implement loading state
```

```
    - Add validation form để add product
    - Responsive design

Scoring:
- Routing: 20 điểm
- Data fetching: 20 điểm
- Form: 20 điểm
- UI/UX: 20 điểm
- Code quality: 20 điểm
```

---

23. Deployment Guide

23.1. Deploy to Vercel (Recommended)

Step-by-step:

```
# 1. Push code lên GitHub
git add .
git commit -m "Ready for deployment"
git push origin main

# 2. Hoặc deploy trực tiếp
npm install -g vercel
vercel

# Follow prompts:
# - Link to existing project? No
# - Project name? nextjs-blog
# - Directory? ./
# - Framework? Next.js
```

Environment Variables:

```
NEXT_PUBLIC_SITE_NAME=NextJS Blog
NEXT_PUBLIC_SITE_URL=https://your-domain.vercel.app
```

23.2. Share với Học viên

Sau khi deploy:

1. Copy deployment URL
2. Share trong Slide/LMS
3. Học viên có thể:
    - Xem demo live
    - Test functionality
    - Reference khi code

Example URLs:

```
Production: https://nextjs-blog-demo.vercel.app
Staging: https://nextjs-blog-staging.vercel.app
Docs: https://github.com/yourname/nextjs-blog
```

---

24. Additional Resources

24.1. Official Documentation

- Next.js Docs
- React 19 Docs
- TypeScript Handbook
- Tailwind CSS

24.2. Video Tutorials (Vietnamese)

- Next.js cơ bản
- App Router explained
- Server Components deep dive

24.3. Practice Projects

1. Todo App với Next.js
2. E-commerce với Stripe
3. Social Media Clone
4. Portfolio Website

24.4. Community

- Next.js Discord
- Reddit: r/nextjs
- Stack Overflow: [nextjs] tag

---

25. Final Checklist for Instructors

Before Class:

- [ ] Demo app deployed và hoạt động
- [ ] Slides prepared
- [ ] Code examples tested
- [ ] Exercises ready
- [ ] GitHub repo setup
- [ ] Environment variables configured

During Class:

- [ ] Live coding setup ready
- [ ] DevTools open trong browser
- [ ] Terminal visible cho học viên
- [ ] Screen recording (nếu cần)
- [ ] Code snippets accessible

After Class:

- [ ] Share deployment URL
- [ ] Push code lên GitHub
- [ ] Post exercises
- [ ] Collect feedback
- [ ] Update materials based on feedback

Note for Claude Code:
Spec này đã được tối ưu hoá để:

1. ✅ Build ứng dụng demo hoàn chỉnh
2. ✅ Hỗ trợ giảng dạy 12h
3. ✅ Cung cấp exercises và solutions
4. ✅ Include teaching notes và troubleshooting
5. ✅ Sẵn sàng deploy và share

Bắt đầu build từ Section 3 (Cấu trúc thư mục) và follow theo từng section!

END OF SPECIFICATION